Project Document: NYC Taxi Trip Duration Prediction

Table of Contents

10. Artifacts

Project Objective

The primary objective of this project is to develop and deploy a machine learning model capable of accurately predicting NYC Taxi Trip Durations. The project encompasses a full MLOps lifecycle, from initial data loading and exploration through feature engineering, model training and comparison, data drift simulation, monitoring, hyperparameter tuning, and finally, model packaging and serving as a REST API. The goal is to build a robust system that can adapt to changing data patterns and provide reliable predictions.

Data Loading and Initial Exploration

Data Source

The project utilizes raw NYC Taxi Trip data from the CSV file NY_Trip_Raw_Data.csv.

Loading Process

The data is loaded into a pandas DataFrame using pd.read_csv().

```
import pandas as pd
df = pd.read_csv('/content/sample_data/NY_Trip_Raw_Data.csv')
print("Data loaded successfully. First 5 rows:")
print(df.head())
```

Initial Exploration

Basic exploratory data analysis (EDA) was performed to understand the dataset's structure and identify potential issues.

- **df.info():** Provided a summary of the DataFrame, including column names, non-null counts, and data types. It showed 1,048,575 entries and identified pickup_datetime and dropoff_datetime as object types, requiring conversion.
- **df.isnull().sum():** Confirmed that there were no missing values across any columns initially.
- **df.describe():** Offered descriptive statistics for numerical columns, revealing insights into ranges for passenger_count, trip_duration, and geographical coordinates. Notably, trip_duration had a wide range, indicating potential outliers.

Data Preprocessing and Feature Engineering

This crucial stage involves cleaning the data, validating its schema, and creating new features to enhance the model's predictive power.

Schema Validation

1. **Datetime Conversion:** pickup_datetime and dropoff_datetime columns were converted from object to datetime format using pd.to_datetime(). No unparseable dates were found.
2. **Illogical Timestamps:** Checked for instances where dropoff_datetime occurred before pickup_datetime. No such illogical entries were found, ensuring data integrity.
3. **Missing GPS Pings:** Confirmed no missing values in geographical coordinate columns.

Feature Engineering

Several new features were created to capture temporal and spatial patterns:

- **pickup_hour:** Extracted the hour of the day from pickup_datetime to capture hourly demand patterns.
- **pickup_dayofweek:** Extracted the day of the week (0 for Monday, 6 for Sunday) from pickup_datetime.
- **is_weekend:** A binary flag (0 for weekday, 1 for weekend) derived from pickup_dayofweek.
- **distance_km:** Calculated the Haversine distance between pickup and dropoff coordinates, representing the trip length in kilometers. This is a critical feature for trip duration prediction.
- **weather (Categorical):** The weather column was converted to a categorical data type for efficient encoding during modeling.

Dataset Versioning

The processed dataset, enriched with new features, was saved as NY_Trip_Processed_Data_v1.csv to ensure reproducibility and track data changes.

```
# Example of feature engineering code
# Hour-of-day
df['pickup_hour'] = df['pickup_datetime'].dt.hour
# Weekday/weekend
df['pickup_dayofweek'] = df['pickup_datetime'].dt.dayofweek # Monday=0, Sunday=6
df['is_weekend'] = (df['pickup_dayofweek'] >= 5).astype(int)
```

```
# Distance (Haversine formula)
# ... (haversine_distance function definition) ...
df['distance_km'] = df.apply(lambda row: haversine_distance(
    row['pickup_latitude'], row['pickup_longitude'],
    row['dropoff_latitude'], row['dropoff_longitude']
), axis=1)
# Weather (convert to categorical for efficiency)
df['weather'] = df['weather'].astype('category')
```

Model Training and Comparison

Two regression models were trained and compared using MLflow for experiment tracking.

Data Preparation for Modeling

- **Feature Selection:** Irrelevant columns (id, pickup_datetime, dropoff_datetime) and the target variable (trip_duration) were dropped from the feature set X.
- **One-Hot Encoding:** Categorical features (weather, store_and_fwd_flag, vendor_id) were one-hot encoded.
- **Train-Test Split:** The dataset was split into 80% training and 20% testing sets using train_test_split with random_state=42 for reproducibility.

Model 1: Linear Regression

- **Implementation:** A standard LinearRegression model from scikit-learn was used.
- **MLflow Tracking:** The model type (Linear Regression), performance metrics (MSE, R-squared), and the trained model object itself were logged to MLflow under the run name "Linear_Regression_Model".
- **Performance:**
    - Mean Squared Error (MSE): 9557237.30
    - R-squared (R2): 0.03

```
with mlflow.start_run(run_name="Linear_Regression_Model"):
    # ... (model training and evaluation code) ...
    mlflow.log_metric("mse", mse_lr)
    mlflow.log_metric("r2", r2_lr)
    mlflow.sklearn.log_model(lr_model, "linear_regression_model")
```

Model 2: Gradient Boosting Regressor

- **Implementation:** A GradientBoostingRegressor from scikit-learn was used with default parameters (n_estimators=100, learning_rate=0.1).
- **MLflow Tracking:** Model type, hyperparameters, metrics, and the model object were logged to MLflow under "Gradient_Boosting_Model".
- **Performance:**
    - Mean Squared Error (MSE): 11025336.11
    - R-squared (R2): -0.12

Model Comparison Summary

Based on the initial evaluations, the Linear Regression model performed better with an R-squared of 0.03 compared to the Gradient Boosting Regressor's R-squared of -0.12. Although both R-squared values are low, indicating limited predictive power for *absolute* trip duration, Linear Regression emerged as the superior choice for a baseline model due to its simplicity, interpretability, and slightly better initial performance.

Data Drift Simulation and Monitoring

Simulating a 'Rush Hour Surge' Data Drift

To test the model's robustness against changes in real-world data, a 'Rush Hour Surge' data drift was simulated. This involved modifying a copy of the dataset (df_drift) to reflect increased activity during typical rush hours (7-9 AM and 4-6 PM):

- **Increased Passenger Counts:** Passenger counts were randomly increased for 30% of rush hour trips.
- **Increased Distances:** distance_km was increased by 5-15% for rush hour trips.
- **Increased Trip Durations:** trip_duration was increased by 10-20% for rush hour trips, simulating congestion.

The drifted data was then preprocessed to match the feature set expected by the model.

Impact on Model Performance

- **Original Linear Regression Performance:** R2: 0.03, MSE: 9557237.30
- **Linear Regression Performance on Drifted Data:** R2: 0.01, MSE: 34809382.12

Observation: The R-squared value significantly decreased, and MSE drastically increased, indicating a substantial drop in model performance due to the simulated data drift.

Retraining Trigger Design

A mechanism was designed to automatically trigger model retraining upon detecting significant performance degradation.

- **Performance Degradation Threshold:** A 10% drop in R-squared from the original deployed model's performance was set as the threshold. If the R-squared on new data falls below 0.0250 (original 0.0278 * (1 - 0.10)), retraining is triggered.
- **Monitoring Logic:** The system continually evaluates the model on incoming data. If r2_on_new_data < threshold_r2, retraining is initiated.
- **Retraining Action:** The process involves fetching new data, re-running the entire pipeline (preprocessing and training), and logging the retrained model to MLflow. For simplicity in this demo, the model was retrained on the original clean training data.
- **Simulated Retraining Result:** After triggering retraining, the model's performance was restored to its original levels (R2: 0.03, MSE: 9557237.30), demonstrating the effectiveness of the retraining strategy.

Improving Model Performance: Hyperparameter Tuning

Given the low R-squared scores, hyperparameter tuning is essential for improving model performance. GridSearchCV was introduced as a method for systematically searching for optimal hyperparameters for the GradientBoostingRegressor.

- **Parameter Grid:** A small param_grid was defined for demonstration, including n_estimators ([100, 200]), learning_rate ([0.05, 0.1]), and max_depth ([3, 5]).
- **Cross-Validation:** GridSearchCV was configured with cv=3 and scoring='r2' to optimize for R-squared.
- **Outcome (Conceptual):** While the full execution of GridSearchCV was commented out due to computational intensity, the process would identify the best_params_ and best_score_, leading to a tuned_gbr_model with potentially improved performance, which would then be logged to MLflow.

Model Packaging and Serving

The best-performing model (Linear Regression) was packaged for serving as a REST API.

Loading the Model

- The run_id of the trained Linear Regression model was retrieved from MLflow.
- The model was loaded using mlflow.pyfunc.load_model(f"runs:/{lr_run_id}/linear_regression_model").

Demonstrate Prediction

A small sample from the X_test dataset was used to demonstrate making predictions with the loaded model.

Serve Model via REST API using MLflow (Self-Contained Docker)

To facilitate deployment, a self-contained Docker image was created that embeds the model artifacts and does not require a running MLflow tracking server at runtime.

1. **requirements.txt:** A requirements.txt file was generated, listing necessary Python packages (mlflow, scikit-learn, pandas, numpy, gunicorn).
2. **Dockerfile:** A Dockerfile was created:
    - Uses python:3.9-slim-buster as the base image.
    - Installs dependencies from requirements.txt.
    - Copies the downloaded MLflow model artifacts (from linear_regression_model_artifacts) into the image.
    - Exposes port 5000.
    - Defines the CMD to serve the model using mlflow models serve from the local path within the container.

Instructions for Deployment:

- **Download:** Download Dockerfile, requirements.txt, and the linear_regression_model_artifacts directory.
- **Build:** docker build -t my-local-mlflow-model .
- **Run:** docker run -p 5000:5000 my-local-mlflow-model
- **Prediction Payload Example**

MLflow UI Access

For centralized experiment tracking and visualization of model runs, an MLflow UI instance was launched and made accessible via ngrok.

- **Setup:** pyngrok was installed, and existing ngrok processes were terminated. The MLflow UI was started in the background on port 5000.
- **Authentication:** ngrok.set_auth_token was used with a provided authentication token.
- **Public URL:** A public URL was generated by ngrok.connect(5000), allowing external access to the MLflow UI.
- **Usage:** Users can navigate to the provided ngrok URL to view experiment runs, compare models, examine logged parameters and metrics, and download artifacts. The MLFLOW_TRACKING_URI can be set to this URL for remote tracking.

Resources

- **Python Libraries:** pandas, numpy, scikit-learn (for LinearRegression, GradientBoostingRegressor, train_test_split, GridSearchCV), matplotlib, seaborn, mlflow, PyPDF2, pyngrok.
- **Tools:** MLflow for experiment tracking and model management, Docker for containerization, ngrok for exposing local services.
- **Data:** NY_Trip_Raw_Data.csv for initial data, NY_Trip_Processed_Data_v1.csv for processed data.

This document provides a comprehensive overview of the NYC Taxi Trip Duration Prediction project, detailing each stage of the MLOps pipeline implemented.


```
{
  "dataframe_split": {
    "columns": [
      "passenger_count", "pickup_longitude", "pickup_latitude", "dropoff_longitude",
      "dropoff_latitude", "pickup_hour", "pickup_dayofweek", "is_weekend",
      "distance_km", "weather_freezing", "weather_rainy", "weather_snowy",
      "weather_sunny", "weather_windy", "store_and_fwd_flag_Y", "vendor_id_2"
    ],
    "data": [
      [1, -73.982155, 40.767937, -
```

73.964630, 40.765602, 17, 0, 0, 1.498521, 0, 0, 0, 1, 0, 0, 1]
    ]
  }
}

10. Artifacts:

Git Hub location: https://github.com/2025paml563-coder/mlminiproject

Demo recording link:
https://drive.google.com/file/d/1oLJOLFiq5OixF_OhNkA1Xx_s6zF1wA3H/view?usp=drive_link

Project documentation: https://github.com/2025paml563-coder/mlminiproject/tree/main/project_document

Versioned data: https://github.com/2025paml563-coder/mlminiproject/blob/main/data.zip

Feature engineered data: https://github.com/2025paml563-coder/mlminiproject/blob/main/data_feature_engineered.zip